Face Tracking

Based on face recognition and face positioning, the following is face tracking combined with the robotic arm.

1. Face Recognition

The most basic task of face recognition is "face detection". You must first "capture" a face to recognize it when comparing with newly captured faces in the future. Here we only provide face detection methods. Interested students can build their own classifiers and face datasets for name recognition. The most common face detection method is using "Haar cascade classifiers". Object detection using Haar feature-based cascade classifiers is an efficient object detection method proposed by Paul Viola and Michael Jones in their 2001 paper "Rapid Object Detection using a Boosted Cascade of Simple Features". This machine learning method trains cascade functions based on a large number of positive and negative images, then uses them to detect objects in other images. Here, we will use it for face recognition. Initially, the algorithm requires a large number of positive class images (face images) and negative class images (images without faces) to train the classifier. Then we need to extract features from them. The good news is that OpenCV has both trainers and detectors. If you want to train your own object classifier, such as cars, airplanes, etc., you can use OpenCV to create one. Here we use a pre-trained classifier. In OpenCV, face detection in static images and real-time videos has similar operations. Simply put, video face detection just reads each frame of image from the camera and then uses the static image detection method for detection. Face detection first requires a classifier:

```
face_cascade=cv2.CascadeClassifier('123.xml')
```

```
    123.xml is Haar cascade data, this xml can be obtained from data/haarcascades
in the OpenCV3 source code. Next, actual face detection is performed through
face_cascade.detectMultiScale(). We cannot directly pass each frame of image
obtained from the camera into .detectMultiScale(), but should first convert the
image to grayscale because face detection requires this color space.

    (Note: Make sure to input the correct location of 123.xml.)
```

- Start Docker

After entering the main board desktop, open the terminal and run the following command to start the corresponding Dofbot container:

```
./Docker_Ros.sh
```

Then, in the Docker terminal, input the following command to start jupyter-lab,

```
cd
jupyter-lab --allow-root
```

Then, in the browser, input the main board's IP address plus :9999

```
IP:9999         // Example: 192.168.1.11:9999
```

Code path: /root/Dofbot_2D/src/dofbot_face_follow/Face_follow.ipynb

2. Code Block Design

- Import header files

```
import Arm_Lib
import cv2 as cv
import threading
from time import sleep
import ipywidgets as widgets
from IPython.display import display
from face_follow import face_follow
```

- Initialize robotic arm position

```
Arm = Arm_Lib.Arm_Device()
joints_0 = [90, 135, 20, 25, 90, 30]
Arm.Arm_serial_servo_write6_array(joints_0, 1000)
```

- Create instance, initialize parameters

```
# Create instance
follow = face_follow()
# Initialize mode
model = 'General'
```

- Create controls

```
button_layout = widgets.Layout(width='250px', height='50px',
align_self='center')
output = widgets.Output()
# Exit control
exit_button = widgets.Button(description='Exit', button_style='danger',
layout=button_layout)
# Image control
imgbox = widgets.Image(format='jpg', height=480, width=640,
layout=widgets.Layout(align_self='center'))
# Space layout
controls_box = widgets.VBox([imgbox, exit_button],
layout=widgets.Layout(align_self='center'))
# ['auto', 'flex-start', 'flex-end', 'center', 'baseline', 'stretch', 'inherit',
'initial', 'unset']
```

- Control

```
def exit_button_Callback(value):
    global model
    model = 'Exit'
#     with output: print(model)
exit_button.on_click(exit_button_Callback)
```

- Main program

```
def camera():
    global model
    # Open camera
    capture = cv.VideoCapture(0)
    # Be executed in loop when the camera is opened normally
    while capture.isOpened():
        try:
            # Read each frame from camera
            _, img = capture.read()
            # Unify image size
            img = cv.resize(img, (640, 480))
            img = follow.follow_function(img)
            if model == 'Exit':
                cv.destroyAllWindows()
                capture.release()
                break
            imgbox.value = cv.imencode('.jpg', img)[1].tobytes()
        except KeyboardInterrupt:capture.release()
```

- Start

```
display(controls_box,output)
threading.Thread(target=camera, ).start()
```

Then you can see the robotic arm following the face movement.

```
The adjustment of the control algorithm PID is explained in the PID algorithm
section. If adjustment is needed, please refer to the PID algorithm for learning.
For detailed face recognition code library, please see
```

/root/Dofbot_2D/src/dofbot_face_follow/face_follow.py